Performance Testing

Date	02 July 2026
Team ID	
Project Name	**Credit Card Approval Prediction**
Maximum Marks	5 Marks

Step 1: Testing Overview :

Field	Details
Testing Tool Used	Jupyter Notebook, Python `time` module, Scikit-learn Performance Metrics
Type of Testing	Load Testing, Performance Testing
Target Module / API	Credit Card Approval Prediction Model (Random Forest Classifier)
Test Environment	Local Machine (Windows 11, Intel Core i5 12th Gen, 16 GB RAM)
Test Date	02 July 2026

Step 2: Test Scenarios :

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load dataset and preprocess records	1	8	Dataset loaded successfully without errors
2	Train Random Forest Model	1	22	Model training completed successfully
3	Predict approval for test dataset	20	5	Predictions generated within acceptable response time

4	Evaluate model performance	20	3	Accuracy and evaluation metrics generated successfully

Step 3: Performance Test Results :

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 sec	**0.18 sec**	Pass	Fast prediction response
2	Response Time (Max)	< 5 sec	**0.84 sec**	Pass	Maximum response well below threshold
3	Throughput (Req/sec)	> 20 Req/sec	**33 Req/sec**	Pass	Stable throughput achieved
4	Error Rate	< 1%	**0.00%**	Pass	No execution errors observed
5	CPU Utilization	< 80%	**46%**	Pass	Efficient CPU usage
6	Memory Utilization	< 80%	**38%**	Pass	Memory consumption within limits

Step 4: Observations & Analysis

Key Findings

- The machine learning pipeline executed successfully under the tested workload.
- Model prediction latency remained below one second.
- Dataset preprocessing completed without memory overflow.
- Random Forest provided stable performance with consistent prediction speed.
- No runtime exceptions or execution failures occurred during testing.

Bottlenecks Identified

- Initial model training requires more execution time compared to prediction.
- Large datasets (>500,000 records) increase preprocessing time.
- Hyperparameter tuning significantly increases total execution time.

Optimization Steps Taken

- Removed unnecessary features before training.
- Encoded categorical variables efficiently using LabelEncoder.
- Used reusable preprocessing functions to minimize redundant computation.
- Selected Random Forest as the final model due to its balance between accuracy and execution speed.
- Saved the trained model using Joblib to eliminate retraining during future predictions.

Step 5: Screenshots / Evidence :

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Screenshot 1: JMeter Summary Report

Screenshot 2: JMeter Graph Results (Response Time Over Time)

The above screenshots show the JMeter Summary Report and Response Time Graph for the load test performed on the target API.